

SMART VEHICULAR SYSTEMS PROJECT

Driver Drowsiness Detection via Steering Monitoring

Contents

1	INTRODUCTION	3
1.1	The project	3
1.2	Motivations	3
2	REQUIREMENTS	5
2.1	Functional requirements	5
2.2	Non-functional requirements	5
3	DESIGN OF PROPOSED SOLUTION	7
3.1	Sensors used	7
3.2	Choice of map	7
3.3	Data collection protocol	8
3.4	Data cleaning	8
4	SOFTWARE DESIGN	10
4.1	Project structure	10
4.2	DrivingMode Protocol (Strategy pattern)	11
4.3	ClientDriver	11
4.4	Pydantic configuration	12
4.5	Mode implementations	12
4.5.1	LoggerMode	12
4.5.2	InferenceMode	12
5	FEATURE EXTRACTION	13
5.1	Windowing	13
5.2	Steering-only features	14
5.2.1	Micro-correction degradation features	14
5.2.2	Overcorrection features	14
5.3	Steering entropy computation	15
5.4	Low/high frequency ratio	15
6	CLASSIFICATION MODEL	19
6.1	Model selection	19
6.2	Evaluation: leave-one-session-out cross-validation	19
7	RESULTS	20
7.1	Dataset summary	20
7.2	Classification performance	20

7.3	Confusion matrices	20
7.4	Per-session majority vote	21
7.5	Feature analysis	21
8	MQTT WARNING SYSTEM	27
8.1	Two-tier architecture	27
8.2	Response latency comparison	27
8.3	MQTT topics	28
8.4	HUD feedback	28
9	TECHNOLOGIES USED	29
9.1	CARLA simulator	29
9.2	Steering input	29
9.3	Machine learning stack	29
9.4	MQTT broker	30
10	TESTING, OBTAINED RESULTS AND PERFORMANCE	31
10.1	Experimental protocol	31
10.2	Global accuracy	31
10.3	Model comparison	31
10.4	Error analysis	32
11	DEPLOYMENT	33
11.1	Pipeline overview	33
11.2	Repository structure	33
11.3	Environment setup with uv	33
11.4	MQTT broker with Docker	34
11.5	Running the complete system	34
12	CONCLUSIONS	36
12.1	Limitations and strengths	36
12.1.1	Limitations	36
12.1.2	Strengths	36
12.1.3	Future developments	37
12.2	Closing remarks	37

Chapter 1

INTRODUCTION

1.1 The project

The following document represents the technical report about our Smart Vehicular Systems project.

Our proposed project is a variation of proposal number 16, driver drowsiness detection via steering monitoring, integrated with a real-time MQTT-based warning system that alerts the driver when signs of drowsiness are detected in the steering signal.

More in detail, the goal of our project is the implementation of an integrated system composed by:

- a data collection framework that logs steering behaviour during manual driving sessions in CARLA at 20 Hz, together with ground-truth drowsiness labels (Karolinska Sleepiness Scale self-reports)
- a feature extraction pipeline that computes 11 literature-standard steering-only features from rolling time windows
- a machine learning classifier (logistic regression) trained with leave-one-session-out cross-validation to distinguish alert from drowsy driving
- a two-tier real-time warning system deployed via MQTT, combining a fast rule-based steering-inactivity detector with the ML model for confirmed drowsiness detection.

1.2 Motivations

We have chosen the development of this project for many reasons:

- attempt to reduce road accidents caused by driver drowsiness and fatigue
- attempt to detect early signs of microsleep episodes before a loss-of-control event occurs
- demonstrate a non-intrusive drowsiness detection approach that requires no additional sensors (camera, EEG, etc.) beyond the steering input already available in any vehicle.

According to recent and well-documented statistics about road safety, driver drowsiness is one of the leading causes of fatal accidents:

- drowsy driving is estimated to cause 10–20% of all road crashes in Europe [1]
- the European Transport Safety Council estimates that fatigue contributes to 20–30% of fatal crashes on motorways [2]
- drowsiness-related crashes tend to be more severe because the driver makes no corrective action before impact
- unlike distraction or impairment, drowsiness builds gradually and the driver is often unaware of their own degradation.

These data pushed us to develop a system that can detect drowsiness from the steering signal alone, without requiring additional hardware, following the global effort in developing useful ADAS (Advanced Driver Assistance Systems) for reducing human error and saving lives.

Chapter 2

REQUIREMENTS

The requirements of the project can be split into two main categories: functional requirements collect what the system does, while non-functional requirements collect which relevant feature the system must have.

2.1 Functional requirements

The main functionalities of the system are:

1. manual driving data collection in CARLA with continuous mouse-based steering at 20 Hz
2. logging of steering angle, vehicle state, lane metrics, and self-reported KSS drowsiness rating per session
3. extraction of 11 steering-only features per time window (15 s windows, 75% overlap)
4. binary classification of each window as alert or drowsy using a logistic regression model
5. leave-one-session-out cross-validation for honest evaluation without data leakage
6. real-time drowsiness prediction during live driving in CARLA
7. two-tier warning system:
 - 7.1 Tier 1: steering inactivity detector that fires within 3 seconds of steering going still at speed (catches microsleep onset)
 - 7.2 Tier 2: ML-based detector that confirms sustained drowsiness from the steering feature pattern
8. publishing of all drowsiness events (state changes, inactivity warnings, probability heartbeats) on an MQTT broker.

2.2 Non-functional requirements

The main features of the system are:

9. **reactiveness** → the inactivity detector operates at 20 Hz (every tick) and fires within 3 seconds; the ML model predicts every 2 seconds with majority-vote smoothing over 2 consecutive predictions and a high-confidence bypass that fires immediately when $P(\text{drowsy}) \geq 0.80$
10. **interpretability** → the logistic regression model provides standardised coefficients that directly indicate which steering features contribute to drowsiness detection, enabling clinical and engineering interpretation
11. **non-intrusiveness** → the system uses only the steering signal, requiring no camera, wearable sensors, or physiological measurements
12. **honest evaluation** → the system is validated using leave-one-session-out cross-validation, ensuring that no data from the test session leaks into training.

Chapter 3

DESIGN OF PROPOSED SOLUTION

3.1 Sensors used

Unlike computer-vision-based drowsiness detection approaches that rely on cameras to monitor the driver’s face (eye closure, yawning, head nodding), our system uses only the steering input signal. This follows the approach used by production systems such as the Mercedes-Benz Attention Assist and the Bosch driver drowsiness detection module, which analyse steering wheel angle patterns to detect fatigue.

In the CARLA simulator, the steering signal is captured via continuous mouse input at 20 Hz in synchronous mode, providing a fixed-timestep, reproducible signal suitable for frequency-domain analysis.

The parameters of the data collection setup are summarised in the table below:

Parameter	Value
Simulator	CARLA 0.9.x, synchronous mode
Sampling rate	20 Hz (fixed delta 0.05 s)
Input device	Mouse (continuous steering) + keyboard (pedals)
Steering signal	<code>steer_cmd</code> $\in [-1, 1]$
Map	Town04 (highway ring, sustained monotonous driving)
Camera (driver view)	RGB, 800×450 (low graphics mode)

Table 3.1: Parameters of the data collection setup.

3.2 Choice of map

The choice of map is critical for this experiment because drowsiness is induced by monotony and the steering metrics only make sense during sustained lane-keeping. Town04 was selected because it contains a large highway ring (a figure-8 loop) that can be driven continuously with gentle banked curves and almost no intersections. This provides the monotonous, sustained-steering condition that both surfaces drowsiness and makes steering-based metrics meaningful.

Urban maps (Town01/02/03/05) were avoided because their frequent intersections and turns fragment the steering signal and force steering behaviours unrelated to drowsiness.

3.3 Data collection protocol

Data was collected across 6 driving sessions from a single subject, split into two conditions:

Session	Label	Windows	Condition
drive_alert_20260415_132532	alert	180	well-rested
drive_alert_20260417_145754	alert	131	well-rested
drive_alert_20260418_190930	alert	183	well-rested
drive_drowsy_20260422_233318	drowsy	192	sleep-deprived
drive_drowsy_20260423_064125	drowsy	166	sleep-deprived
drive_drowsy_20260426_055744	drowsy	194	sleep-deprived
Total		1046	

Table 3.2: Summary of the 6 driving sessions collected.

Each session was labelled at the session level as either alert or drowsy. During driving, the subject reported their current Karolinska Sleepiness Scale (KSS) rating (1–9) via keypress every few minutes.

The following data was logged at 20 Hz per tick: wall clock time, simulation time, frame number, raw steering angle, applied steering command, throttle, brake, reverse flag, vehicle speed, vehicle position and heading, cross-track error from lane centre, heading error, lane width, road curvature, junction flag, KSS rating, and session metadata.

3.4 Data cleaning

A critical design decision was the cleaning strategy. Initially, aggressive per-row filtering was applied (dropping rows on junctions, in reverse, at low speed, or on sharp curves), but this fragmented the continuous steering signal and destroyed too many usable windows.

After analysis, we adopted a minimal cleaning approach: the only filter applied is the **junction flag** (`on_junction == 1`), since intersection driving forces turns unrelated to drowsiness. Speed drops, reverse manoeuvres, and curvature spikes are **deliberately kept** because they are symptoms of drowsy overcorrection and loss of control precisely the events that carry the strongest drowsiness signal.

Windows are accepted if at least 70% of their rows are not on a junction. The steering signal within each window is linearly detrended before computing micro-correction features, so that the gentle curvature of the Town04 ring does not contaminate the variability metrics.

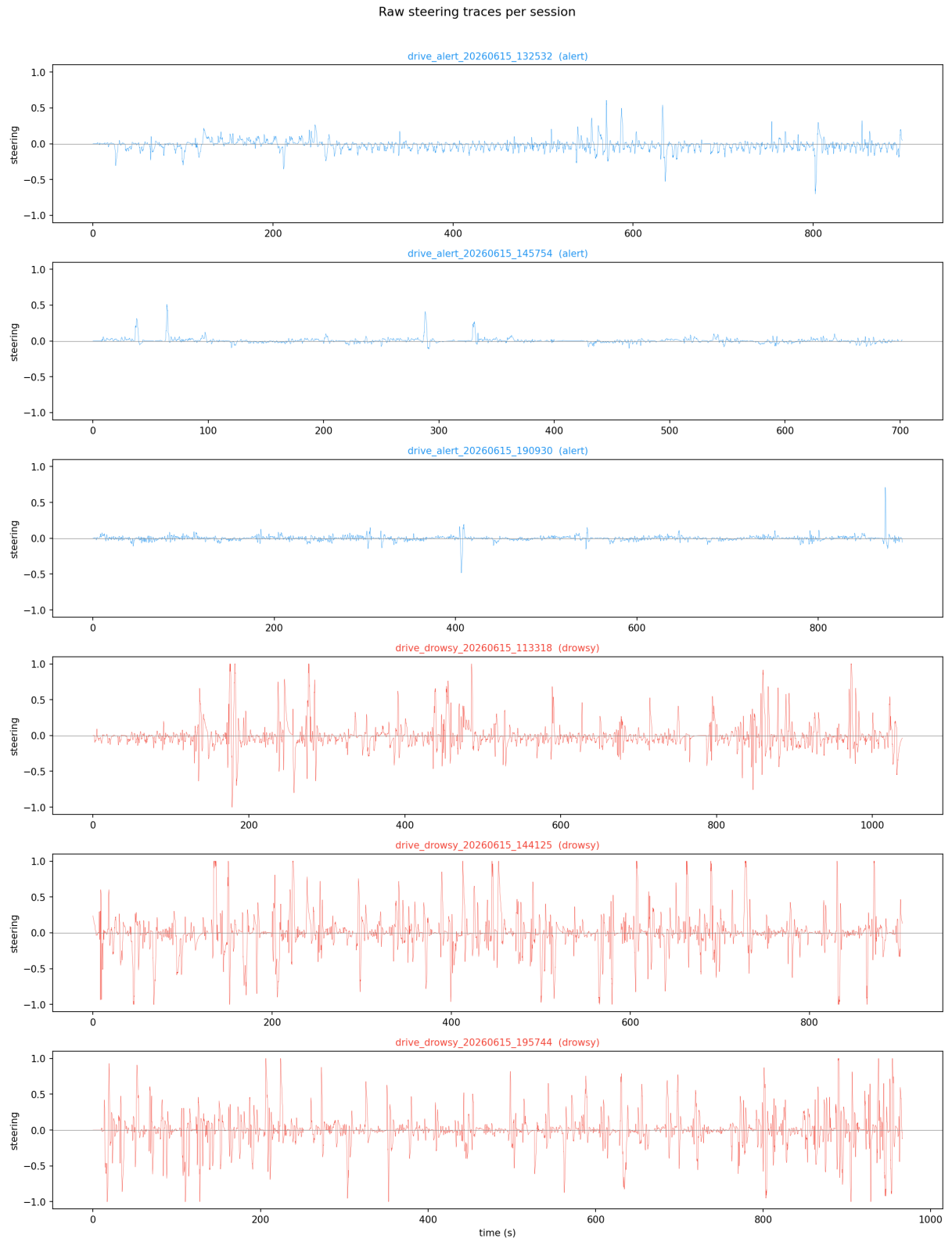


Figure 3.1: Raw steering traces for all 6 sessions. Alert sessions (blue) show steady micro-corrections; drowsy sessions (red) show periods of inactivity followed by large over-corrections.

Chapter 4

SOFTWARE DESIGN

The codebase follows a modular, typed architecture designed around separation of concerns: the driving infrastructure (CARLA, pygame, rendering) is decoupled from mode-specific behaviour (logging vs. live inference) via a Protocol-based strategy pattern. All configuration is validated at startup through Pydantic models, and every function signature carries explicit type annotations.

4.1 Project structure

The project is organised as a Python package with relative imports and `__init__.py` files in every directory:

Listing 4.1: Project directory structure.

```
1 app/
2   __init__.py
3   config.py           # Pydantic configuration models
4   feature_extraction.py # 11 steering features (shared)
5   client_driver.py    # ClientDriver class
6   run_logger.py       # Entry point: data collection
7   run_inference.py    # Entry point: live detection
8   modes/
9       __init__.py
10      driving_mode.py  # DrivingMode Protocol + WarningState
11      logger_mode.py   # LoggerMode (CSV writing)
12      inference_mode.py # InferenceMode (ML + MQTT)
13   scripts/
14       __init__.py
15       extract_features.py # Offline feature extraction + plots
16       train_models.py    # Model training + evaluation + plots
17   utils/
18       __init__.py
19       lane_metrics.py    # LaneMetrics dataclass
20       mqtt_warner.py     # MQTTWarner class
```

All scripts are invoked as modules from the parent directory using `python -m app.run_logger`, `python -m app.scripts.extract_features`, etc., which ensures the relative import chain

resolves correctly through the `__init__.py` hierarchy.

4.2 DrivingMode Protocol (Strategy pattern)

The central design pattern is a `@runtime_checkable Protocol` that defines the interface between the shared driving loop (`ClientDriver`) and mode-specific behaviour. Any mode must implement six typed hooks:

Hook	Responsibility
<code>on_setup(driver)</code>	One-time initialisation after CARLA connects (open CSV, allocate buffers)
<code>on_key(key, driver) -> bool</code>	Handle mode-specific keypresses (KSS rating, markers); return <code>True</code> if consumed
<code>on_tick(driver, snapshot)</code>	Per-tick logic at 20 Hz (write CSV row, run prediction, publish MQTT)
<code>get_hud_lines(driver) -> list[str]</code>	Return 1–3 text lines for the mode-specific HUD overlay
<code>get_warning_state() -> WarningState</code>	Return the current warning type and remaining ticks for the visual overlay
<code>on_cleanup()</code>	Release resources on shutdown (close files, disconnect MQTT)

Table 4.1: DrivingMode Protocol hooks.

Using a Protocol rather than an abstract base class means modes satisfy the interface structurally (duck typing checked at lint time by `mypy/pyright` and at runtime via `isinstance`), with no inheritance coupling. Adding a new mode (e.g. autopilot baseline, replay mode) requires only implementing these six methods.

4.3 ClientDriver

`ClientDriver` owns all shared infrastructure: CARLA connection, world/map setup, Traffic Manager, vehicle spawning, NPC traffic, chase camera, pygame display, mouse steering, keyboard pedals, and the render loop. It is completely mode-agnostic it receives a `DrivingMode` instance at construction and calls the Protocol hooks at well-defined points in the loop:

Listing 4.2: Simplified `ClientDriver` main loop.

```

1 def run(self) -> None:
2     self.mode.on_setup(self)
3     while running:
4         self.world.tick()
5         # handle events -> self.mode.on_key(...)
6         self._update_controls()
7         self._update_vehicle_state()
8         self.mode.on_tick(self, snapshot)
9         self._render() # calls mode.get_warning_state()
10                        # and mode.get_hud_lines()

```

4.4 Pydantic configuration

All tuneable parameters are centralised in Pydantic `BaseModel` classes with field-level validation and documentation:

Model	Fields	Purpose
<code>SimConfig</code>	<code>hz</code>	Simulation tick rate
<code>SteeringConfig</code>	<code>sensitivity</code> , <code>autocenter</code>	Mouse feel (validated $0 \leq \text{autocenter} \leq 1$)
<code>InactivityConfig</code>	<code>steer_thresh</code> , <code>time_s</code> , <code>min_speed_kmh</code>	Tier-1 warning thresholds
<code>FeatureConfig</code>	<code>reversal gap</code> , <code>frequency bands</code> , <code>entropy bins</code>	Feature extraction constants
<code>ExtractionMeta</code>	<code>alpha</code> , <code>fs</code> , <code>window_s</code> , <code>overlap</code>	Saved with <code>features.csv</code> for reproducibility
<code>ModelMeta</code>	<code>feature_cols</code> , <code>classes</code> , <code>alpha</code> , <code>fs</code>	Saved with the pickle for live inference

Table 4.2: Pydantic configuration models.

A module-level singleton `CONFIG = AppConfig()` is imported wherever needed. Invalid values (e.g. `autocenter=1.5`) are rejected at startup with a clear Pydantic validation error rather than producing silent bugs at runtime.

4.5 Mode implementations

4.5.1 LoggerMode

Implements `DrivingMode` for data collection. On each tick it computes lane metrics via `compute_lane_metrics()`, writes a CSV row, and handles KSS keypress events. The CSV header and column order are defined as a module-level typed constant. A 3-minute KSS reminder timer nudges the driver to update their self-reported sleepiness.

4.5.2 InferenceMode

Implements `DrivingMode` for live detection. It maintains a rolling deque of steering samples, runs the Tier-1 inactivity check on every tick, and triggers the Tier-2 ML prediction every 2 seconds. The high-confidence bypass ($P \geq 0.80$) fires immediately; borderline predictions go through the 2-window majority vote. All state changes and heartbeats are published via `MQTTWarner`.

Chapter 5

FEATURE EXTRACTION

5.1 Windowing

The continuous steering signal is sliced into fixed-length windows of 15 seconds (300 samples at 20 Hz) with 75% overlap (step of 2 s). This configuration balances temporal resolution against window length: 15 seconds is long enough to capture a full drift-then-correct cycle (the characteristic drowsiness pattern), while the high overlap maximises the number of training examples from limited session data.

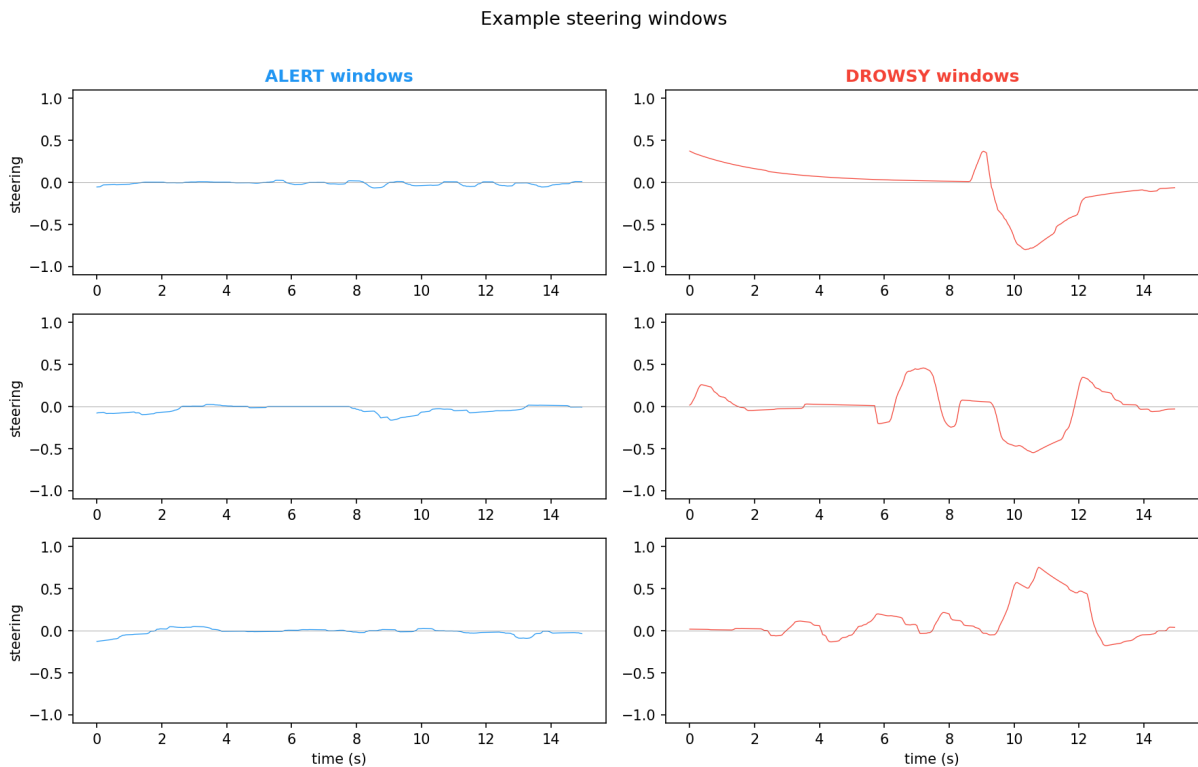


Figure 5.1: Example 15s steering windows. Alert windows (left, blue) show continuous micro-corrections; drowsy windows (right, red) show flat drift periods and sudden over-corrections.

5.2 Steering-only features

All 11 features are computed solely from the steering signal. They are grouped into two categories that capture different aspects of drowsy driving behaviour.

5.2.1 Micro-correction degradation features

These capture the subtle, sustained degradation of the driver’s continuous lane-keeping corrections:

Feature	Description
<code>steer_sd</code>	Standard deviation of steering angle (overall variability)
<code>steer_range</code>	Peak-to-peak steering angle range
<code>steer_vel_sd</code>	Standard deviation of steering velocity (rate-of-change variability)
<code>steer_vel_mean_abs</code>	Mean absolute steering velocity (average correction intensity)
<code>reversal_rate</code>	Direction reversals exceeding a dead-band per minute (micro-correction frequency)
<code>hold_fraction</code>	Fraction of window where steering velocity ≈ 0 (driver stopped correcting)
<code>lf_hf_ratio</code>	Ratio of low-frequency (0–0.2 Hz) to high-frequency (0.2–2 Hz) steering power via Welch PSD
<code>steering_entropy</code>	Nakayama/Boyle steering entropy: unpredictability of steering relative to a second-order predictor

Table 5.1: Micro-correction degradation features.

5.2.2 Overcorrection features

These capture the dramatic events that follow a microsleep the violent jerk back to the lane:

Feature	Description
<code>max_abs_steer_rate</code>	Maximum absolute steering velocity in the window (hardest single jerk)
<code>steer_rate_kurtosis</code>	Excess kurtosis of steering velocity distribution (heavy tails = rare extreme corrections)
<code>large_correction_rate</code>	Number of times per minute steering velocity exceeds a threshold (frequency of big jerks)

Table 5.2: Overcorrection features.

5.3 Steering entropy computation

The steering entropy follows the Nakayama/Boyle method. A second-order Taylor predictor is applied to the detrended steering signal:

$$\hat{s}[n] = 3s[n-1] - 3s[n-2] + s[n-3]$$

The prediction errors $e[n] = s[n] - \hat{s}[n]$ are binned into 9 categories defined by multiples of a scale factor α (the 90th percentile of $|e|$ computed from the alert baseline). The Shannon entropy of the resulting distribution, normalised by $\log 9$, gives a value in $[0, 1]$ where higher values indicate coarser, less predictable steering.

5.4 Low/high frequency ratio

The steering power spectral density is estimated via Welch's method. Power is integrated over the low-frequency band (0–0.2 Hz, corresponding to slow drift and lazy corrections) and the high-frequency band (0.2–2 Hz, corresponding to alert micro-corrections). Drowsy driving shifts energy from the HF band to the LF band, increasing the ratio.

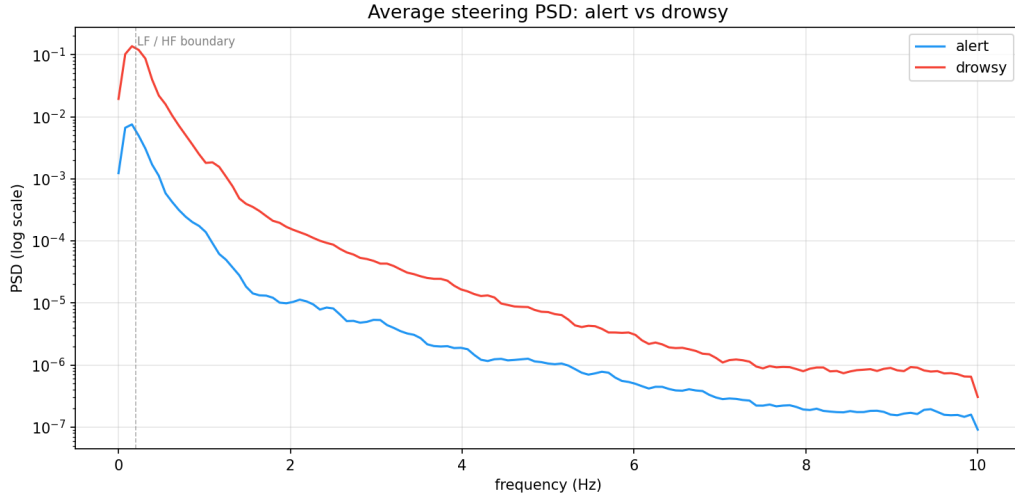


Figure 5.2: Average steering power spectral density. Drowsy driving (red) shows higher low-frequency power (drift) and reduced high-frequency micro-corrections compared to alert driving (blue). The dashed line marks the LF/HF boundary at 0.2 Hz.

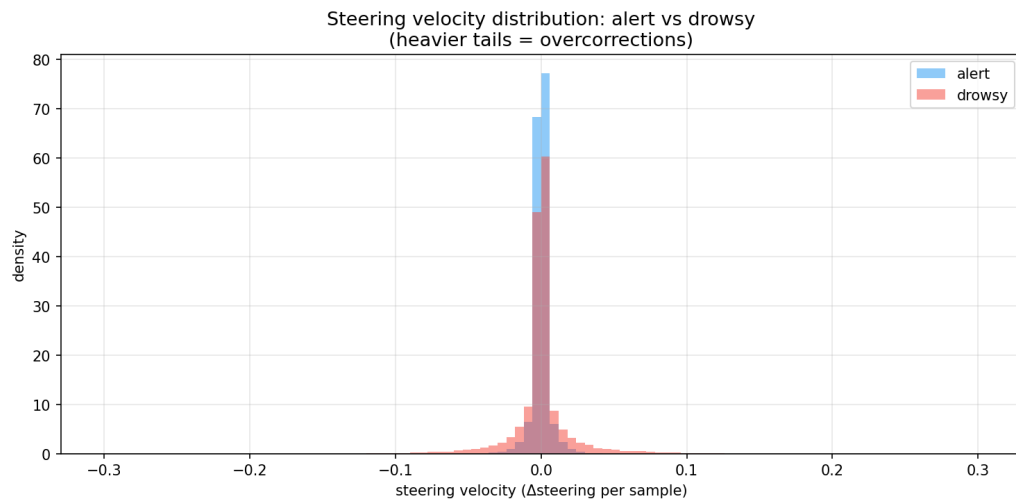


Figure 5.3: Steering velocity distribution. Drowsy driving (red) has heavier tails than alert driving (blue), reflecting the rare but extreme overcorrection events.

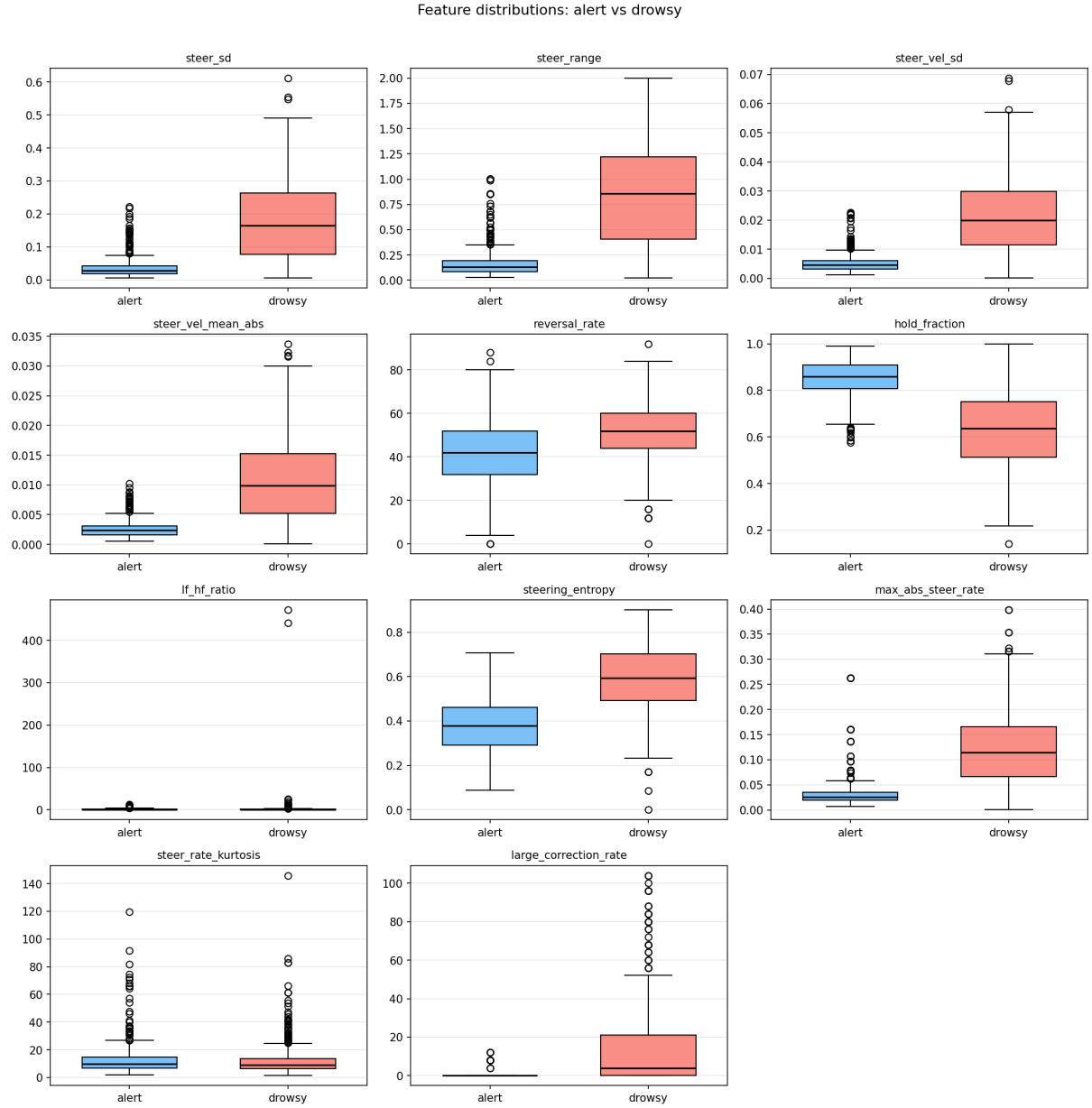


Figure 5.4: Feature distributions: alert vs drowsy. Box plots for all 11 steering features show separation between classes.

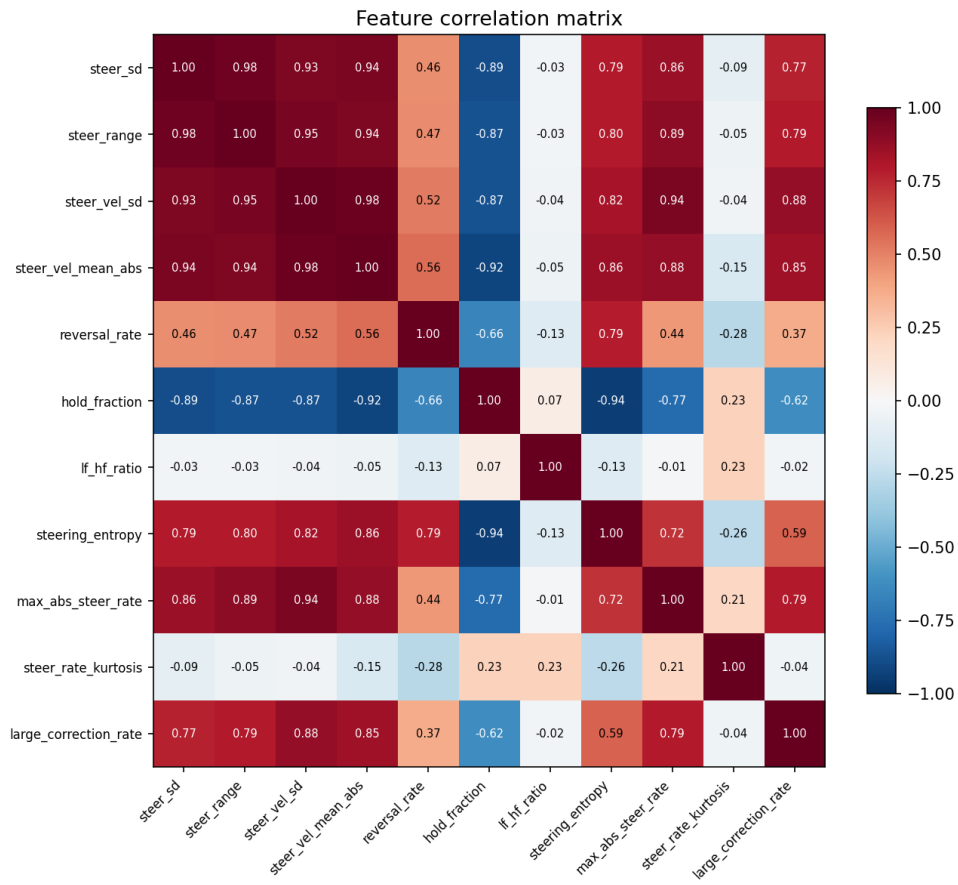


Figure 5.5: Feature correlation matrix. Highly correlated feature pairs (e.g. `steer_sd` and `steer_range`) share variance but capture different aspects of the steering signal.

Chapter 6

CLASSIFICATION MODEL

6.1 Model selection

Two classifiers were evaluated:

- **Logistic regression** (with standardised features and balanced class weights) as the interpretable baseline. Its coefficients directly indicate which steering features contribute to drowsiness and in which direction.
- **Random forest** (300 trees, max depth 4, min samples leaf 3, balanced subsample weighting) as a capacity-limited ensemble.

Deep learning approaches (LSTMs, 1D-CNNs) were deliberately avoided. With approximately 1000 correlated windows from a single subject, a neural network would memorise the subject and session rather than learning drowsiness.

6.2 Evaluation: leave-one-session-out cross-validation

The critical methodological choice is the evaluation split. Windows from the same driving session are highly correlated (overlapping, same route, same driver state). A random train/test split would place correlated windows on both sides and produce inflated accuracy.

Instead, we use **leave-one-session-out (LOSO) cross-validation**: in each of the 6 folds, one entire session is held out for testing and the remaining 5 sessions are used for training. Every window is predicted exactly once, by a model that never saw any data from that window's session. Metrics are computed on the pooled out-of-fold predictions.

Chapter 7

RESULTS

7.1 Dataset summary

Property	Value
Total windows	1046
Alert windows	494
Drowsy windows	552
Number of sessions	6 (3 alert, 3 drowsy)
Number of features	11 (steering-only)
Window length	15 s (300 samples)
Window overlap	75%
Sampling rate	20 Hz

Table 7.1: Dataset summary.

7.2 Classification performance

Metric	Logistic Regression	Random Forest
Accuracy	0.793	0.719
Balanced accuracy	0.794	0.716
Precision	0.830	0.718
Recall	0.763	0.770
F1 score	0.795	0.743
ROC-AUC	0.850	0.833
PR-AUC	0.871	0.860

Table 7.2: Classification performance with leave-one-session-out cross-validation.

7.3 Confusion matrices

Logistic regression confusion matrix (pooled out-of-fold predictions):

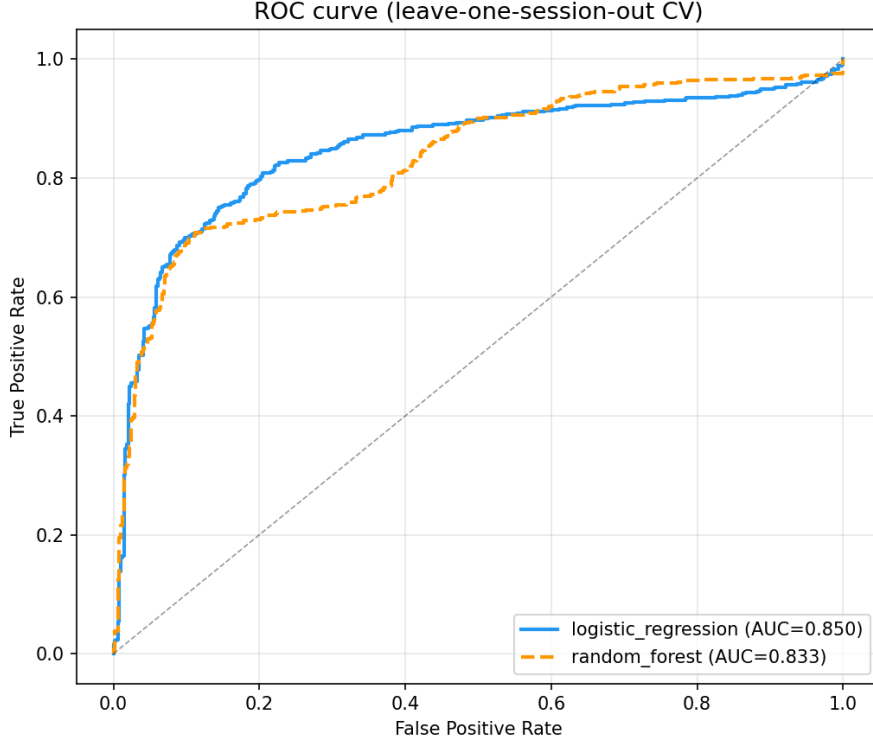


Figure 7.1: ROC curves for both models. Logistic regression achieves a higher AUC (0.850 vs 0.833), demonstrating better discrimination at all thresholds.

	Predicted alert	Predicted drowsy
True alert	408	86
True drowsy	131	421

Table 7.3: Confusion matrix for logistic regression (LOSO CV).

7.4 Per-session majority vote

Beyond window-level metrics, each session was classified by majority vote over its windows. The logistic regression correctly classified all 6 sessions:

The random forest achieved 5/6, misclassifying one alert session (alert_132532), which was driven with a different route pattern than the other alert sessions.

7.5 Feature analysis

The logistic regression coefficients (standardised, fitted on all data) reveal which steering features are most predictive of drowsiness:

The dominant feature is `steer_vel_sd` (+3.70): drowsy drivers alternate between periods of no steering correction (during drift) and sudden violent corrections (during over-correction), producing high variability in the steering rate. The positive coefficient on `lf_hf_ratio` (+0.30) matches the literature prediction that drowsy steering energy shifts from the high-frequency micro-correction band to the low-frequency drift band.

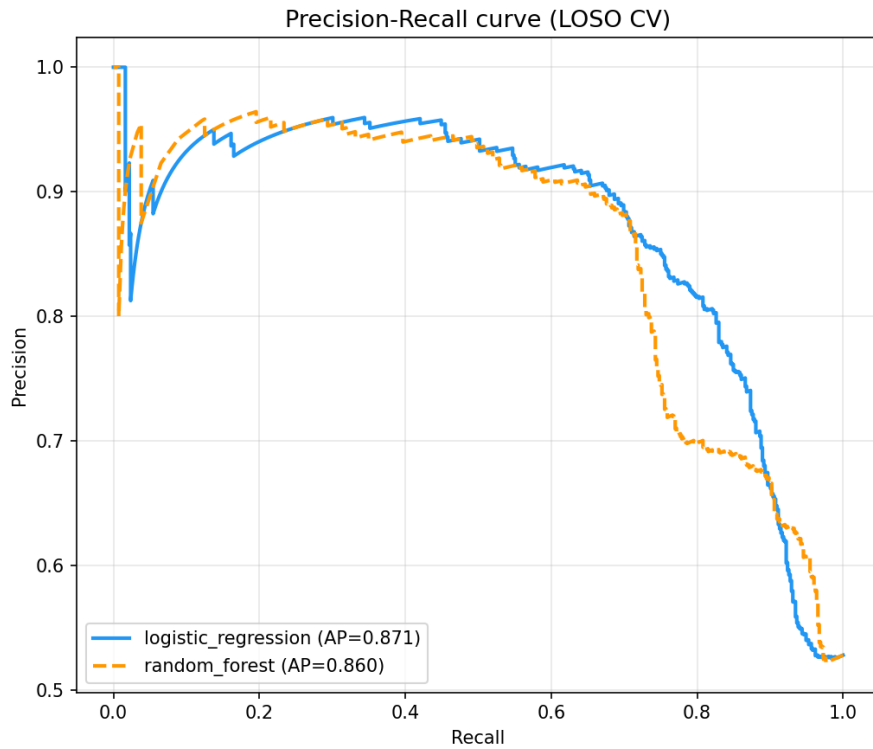


Figure 7.2: Precision-Recall curves. The logistic regression maintains higher precision across recall levels, reflected in its superior average precision (0.871 vs 0.860).

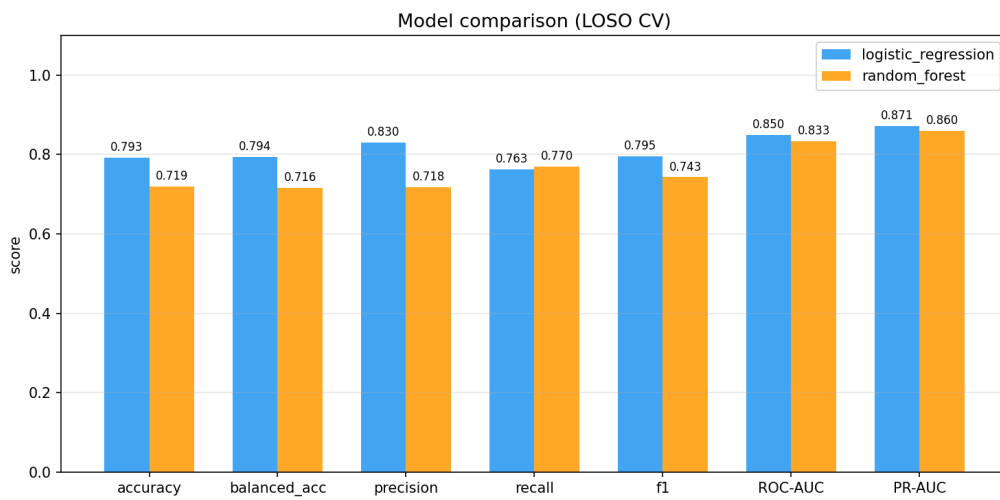


Figure 7.3: Side-by-side comparison of all evaluation metrics for both models.

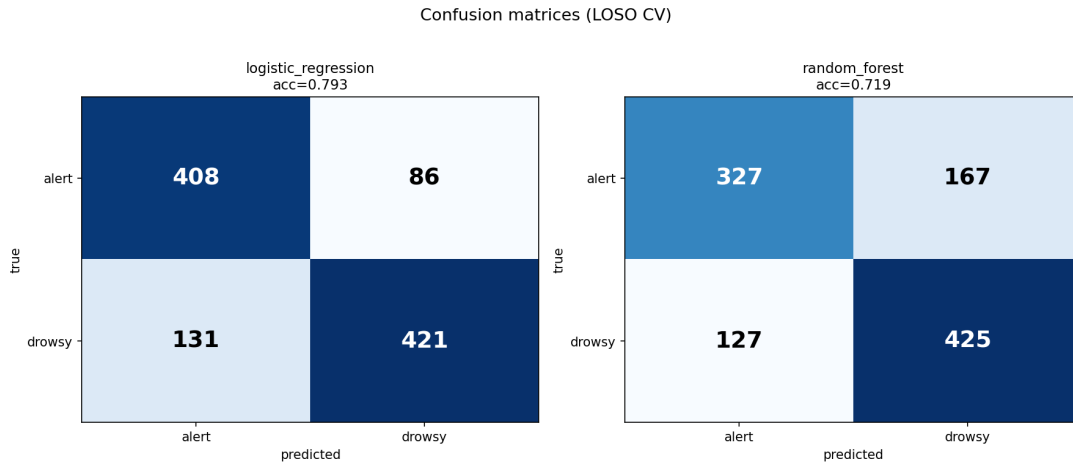


Figure 7.4: Confusion matrices for both models. The logistic regression has fewer false positives (86 vs 167), resulting in higher precision.

Session	True label	Predicted	Correct
alert_132532	alert	alert	
alert_145754	alert	alert	
alert_190930	alert	alert	
drowsy_113318	drowsy	drowsy	
drowsy_144125	drowsy	drowsy	
drowsy_195744	drowsy	drowsy	
Sessions correct			6/6

Table 7.4: Per-session majority vote results (logistic regression).

Feature	Coefficient (+ = more drowsy)
steer_vel_sd	+3.700
steer_vel_mean_abs	+1.235
large_correction_rate	−1.142
steer_sd	−0.926
hold_fraction	−0.738
max_abs_steer_rate	+0.514
steering_entropy	−0.504
steer_rate_kurtosis	−0.303
lf_hf_ratio	+0.298
reversal_rate	−0.161
steer_range	+0.100

Table 7.5: Logistic regression coefficients (standardised).

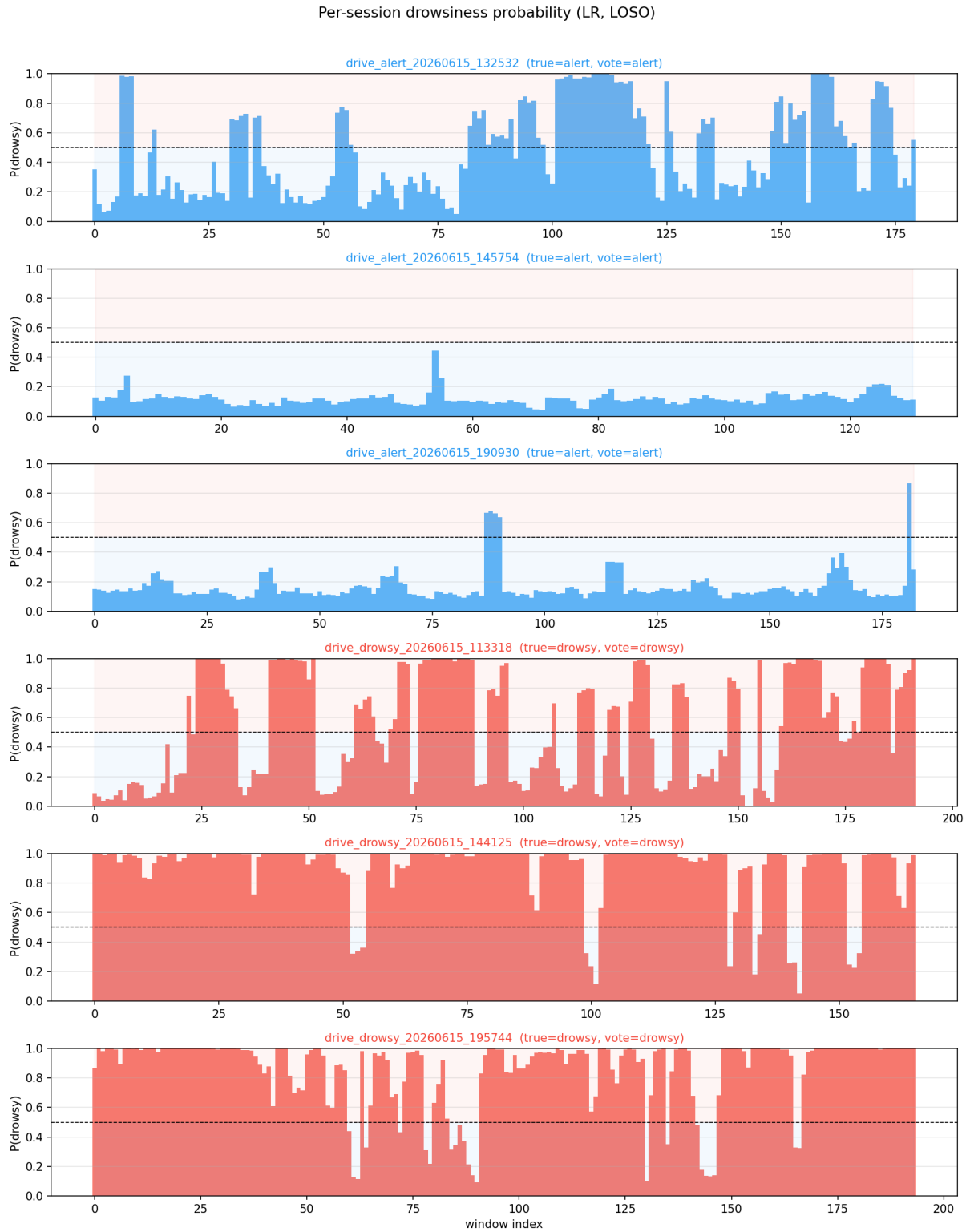


Figure 7.5: Per-session drowsiness probability over time (logistic regression, LOSO). Each bar is one window's $P(\text{drowsy})$. Alert sessions (blue) stay mostly below the 0.5 threshold; drowsy sessions (red) are consistently above it.



Figure 7.6: Per-session feature means. Blue = alert, red = drowsy. Consistent separation across sessions indicates the features capture genuine drowsiness rather than session-specific artefacts.

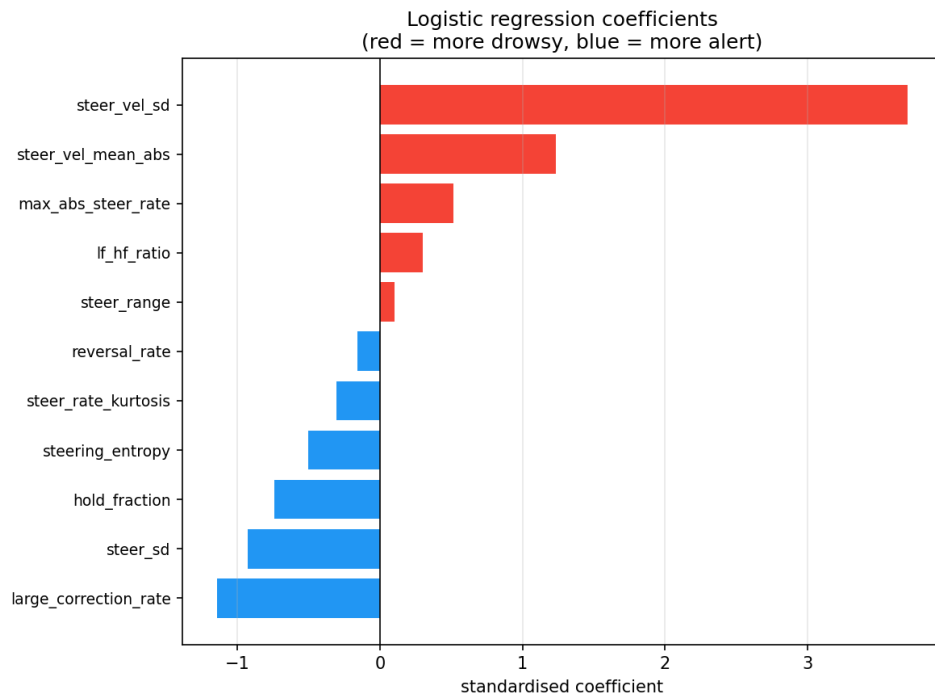


Figure 7.7: Logistic regression coefficients (standardised). Red bars indicate features whose increase is associated with drowsiness; blue bars with alertness.

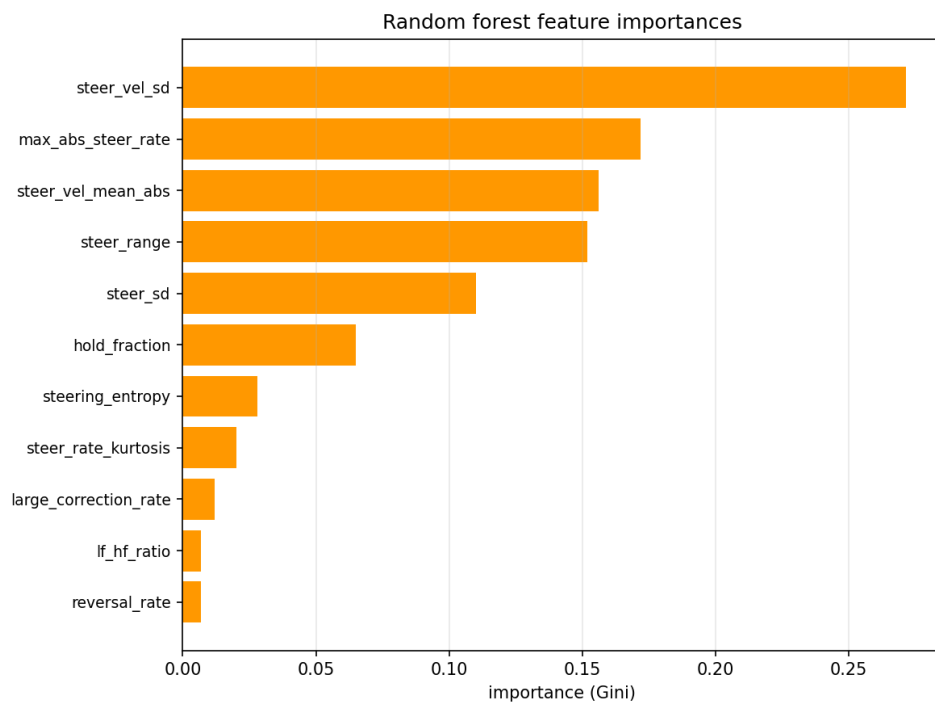


Figure 7.8: Random forest feature importances (Gini). **steer_vel_sd** is the most important feature for both models.

Chapter 8

MQTT WARNING SYSTEM

8.1 Two-tier architecture

A key observation from the model analysis is that the ML classifier fires *after* a drowsy overcorrection event, because the features that dominate the model (`steer_vel_sd`, `max_abs_steer_rate`) require the jerk to have already happened. A real safety system should ideally detect the onset of a microsleep *before* the overcorrection.

To address this, we implement a two-tier warning system with a high-confidence bypass:

- **Tier 1: Steering inactivity detector (fast, rule-based)** → checked every tick at 20 Hz. If the steering has barely moved for more than 3 seconds while the vehicle is travelling above 15 km/h, an early warning fires immediately. An alert driver at highway speed is always making small micro-corrections; steering going still is inherently suspicious and indicative of a microsleep. No ML is needed for this rule.
- **Tier 2: ML-confirmed drowsiness (pattern-based)** → the logistic regression model runs on a rolling 15 s window every 2 seconds. Two decision paths exist:
 - **High-confidence bypass:** if a single prediction yields $P(\text{drowsy}) \geq 0.80$, the warning fires immediately without waiting for the majority vote. This reduces worst-case latency from 11 s to 2 s for clear drowsy episodes.
 - **Smoothed vote:** for borderline predictions ($0.50 \leq P < 0.80$), a majority vote over 2 consecutive predictions is required, giving a worst-case latency of ~ 4 s. This prevents a single noisy window from producing a false alarm.

8.2 Response latency comparison

The high-confidence bypass was introduced after observing that the initial implementation (3 predictions, 2 s intervals) responded too slowly the warning fired *after* the overcorrection was complete rather than during the drowsy episode. The table below summarises the improvement:

Scenario	Initial	Optimised	Improvement
High confidence ($P \geq 0.80$)	7–11 s	2 s	$\sim 5\times$
Moderate confidence ($0.5 \leq P < 0.8$)	7–11 s	4 s	$\sim 2\times$
Tier 1 inactivity (steering still)	3 s	3 s	unchanged

Table 8.1: Warning latency comparison between initial and optimised implementations.

8.3 MQTT topics

Events are published to a local MQTT broker (Mosquitto, port 1883) under the following topics:

Topic	Description
carla/drowsiness/state	Retained message: “ALERT”, “INACTIVITY”, or “DROWSY” (on state change)
carla/drowsiness/inactivity	JSON: inactive seconds, speed (Tier 1 early warning, fires during microsleep)
carla/drowsiness/warning	JSON: drowsy probability, consecutive count (Tier 2 ML confirmed)
carla/drowsiness/heartbeat	JSON: current state + probability (every prediction cycle)

Table 8.2: MQTT topics used by the warning system.

8.4 HUD feedback

The driver receives visual feedback on the CARLA rendering window:

- **Green HUD** $\rightarrow$ ALERT state, normal driving
- **Yellow border** + “WAKE UP!” $\rightarrow$ Tier 1 inactivity warning, fires within 3 seconds of steering going still
- **Red border** + “!! DROWSY !!” $\rightarrow$ Tier 2 ML-confirmed drowsiness, overrides yellow warning.

Chapter 9

TECHNOLOGIES USED

9.1 CARLA simulator

CARLA is an open-source autonomous driving simulator built on Unreal Engine. We used CARLA in synchronous mode with a fixed timestep of 0.05 s (20 Hz), ensuring reproducible and fixed-rate steering sampling. The server was launched in low-quality mode (`-quality-level=Low`) to reduce GPU load.

9.2 Steering input

Due to the unavailability of a physical steering wheel, continuous mouse-based steering was used. The mouse provides a continuous steering signal in $[-1, 1]$ via relative horizontal displacement, with a configurable sensitivity and light auto-centring spring. While not identical to a physical steering wheel, mouse input preserves the continuous motor control signal that carries the drowsiness information, unlike keyboard steering which produces only discrete on/off inputs.

9.3 Machine learning stack

Library	Purpose
NumPy, Pandas	Data handling and feature computation
SciPy	Welch PSD estimation, signal detrending
scikit-learn	Logistic regression, random forest, LOSO CV, metrics
Pydantic	Configuration validation, structured metadata models
Matplotlib	All plots (14 figures for report)
paho-mqtt	MQTT client for publishing warnings
Pygame	CARLA rendering and input handling

Table 9.1: Python libraries used.

9.4 MQTT broker

We used Mosquitto as a local MQTT broker on port 1883. Subscribers can monitor all drowsiness events in real time via:

Listing 9.1: Subscribing to drowsiness warnings.

```
1 mosquitto_sub -t "carla/drowsiness/#" -v
```

Chapter 10

TESTING, OBTAINED RESULTS AND PERFORMANCE

10.1 Experimental protocol

Each driving session lasted approximately 15–20 minutes on the Town04 highway ring. Alert sessions were recorded when the subject was well-rested (8+ hours of sleep), and drowsy sessions were recorded when the subject was sleep-deprived (4–5 hours of sleep) or fatigued (late night, early morning). The mouse sensitivity and auto-centre parameters were kept fixed across all sessions to avoid confounding the steering signal.

10.2 Global accuracy

During evaluation via leave-one-session-out cross-validation, 1046 windows were classified:

Metric	Score (Logistic Regression)
Accuracy	0.793
Balanced accuracy	0.794
Precision	0.830
Recall	0.763
F1 score	0.795
ROC-AUC	0.850
PR-AUC	0.871
Sessions correct	6/6

Table 10.1: Global performance metrics.

10.3 Model comparison

The logistic regression consistently outperformed the random forest across all metrics, demonstrating that a simpler, lower-capacity model is better suited to this small, correlated dataset:

Metric	Logistic Reg.	Random Forest
Accuracy	0.793	0.719
ROC-AUC	0.850	0.833
Sessions correct	6/6	5/6

Table 10.2: Model comparison summary.

10.4 Error analysis

The logistic regression produced 217 errors on 1046 windows (20.7% error rate):

Error type	Count	% of total windows
False positive (alert $\rightarrow$ predicted drowsy)	86	8.2%
False negative (drowsy $\rightarrow$ predicted alert)	131	12.5%

Table 10.3: Error breakdown.

The higher false negative rate (12.5%) reflects windows where drowsy driving was relatively stable (no overcorrection occurred in the window), making it indistinguishable from alert driving based on steering alone. This is mitigated by the Tier 1 inactivity detector, which catches the quiet microsleep periods that the ML model misses.

Chapter 11

DEPLOYMENT

11.1 Pipeline overview

The complete system is deployed as a Python package (`app/`) managed by `uv`, with the MQTT broker running in Docker. Four entry points cover the full workflow:

1. `app.run_logger` drive in CARLA and collect labelled steering data via `LoggerMode`
2. `app.scripts.extract_features` compute 11 steering features per window from session CSVs, generate 7 data exploration plots
3. `app.scripts.train_models` train and evaluate classifiers with LOSO CV, generate 7 evaluation plots, save the model and metadata
4. `app.run_inference` drive in CARLA with real-time two-tier drowsiness detection via `InferenceMode` and MQTT warnings.

11.2 Repository structure

Listing 11.1: Top-level repository structure.

```
1 project/  
2   app/                                # Python package (source code)  
3   docker/  
4     docker-compose.yml               # MQTT broker service  
5     mosquitto/  
6       mosquitto.conf                 # Mosquitto configuration  
7   logs/                              # Driving session CSVs  
8   plots/                             # Generated plots (14 figures)  
9   pyproject.toml                     # Dependencies + entry points  
10  uv.lock                             # Locked dependency versions  
11  README.md
```

11.3 Environment setup with uv

Listing 11.2: One-time environment setup using uv.

```
1 # Install uv (if not already installed)
2 pip install uv
3
4 # Create virtual environment and install all dependencies
5 uv venv
6 uv pip install -e .
7
8 # Install CARLA Python API (not on PyPI, ships with simulator)
9 uv pip install path/to/carla-0.9.xx-cpXX-win_amd64.whl
10
11 # Lock dependencies for reproducibility
12 uv lock
```

The `pyproject.toml` also defines convenience entry points, so scripts can be invoked either as modules or as commands:

Listing 11.3: Entry point shortcuts defined in `pyproject.toml`.

```
1 uv run python -m src.scripts.run_logger      --session alert --sleep
   8 --town Town04 --low
2 uv run python -m src.scripts.extract_features --data ./logs --out
   features.csv
3 uv run python -m src.scripts.train_models     --features features.
   csv
4 uv run python -m src.scripts.run_inference --town Town04 --low
```

11.4 MQTT broker with Docker

The MQTT broker is deployed as a Docker container using Docker Compose:

Listing 11.4: Starting and monitoring the MQTT broker.

```
1 # Start the broker
2 cd docker
3 docker compose up -d
4
5 # Subscribe to drowsiness events (from inside the container)
6 docker exec -it drowsy-mosquitto mosquitto_sub -t "carla/drowsiness
   /#" -v
7
8 # Stop the broker
9 docker compose down
```

The Mosquitto container exposes port 1884 for the MQTT protocol. The Python scripts connect to `localhost:1884` by default.

11.5 Running the complete system

The full demo requires three terminals:

Listing 11.5: Complete deployment workflow.

```
1 # Terminal 1: start MQTT broker
2 cd docker && docker compose up -d
3
4 # Terminal 2: subscribe to warnings
5 docker exec -it drowsy-mosquitto mosquitto_sub -t "carla/drowsiness
  /#" -v
6
7 # Terminal 3: drive with live detection
8 cd ..
9 uv run python -m src.scripts.run_inference --town Town04 --low
```

For data collection, replace the Terminal 3 command with:

Listing 11.6: Data collection mode.

```
1 uv run python -m src.scripts.run_logger --session drowsy --sleep
  4.5 --town Town04 --low
```

After collecting all sessions, the offline pipeline runs without CARLA or Docker:

Listing 11.7: Offline training pipeline.

```
1 uv run python -m src.scripts.extract_features --data ./logs --out
  features.csv
2 uv run python -m src.scripts.train_models --features features.csv
```

Chapter 12

CONCLUSIONS

12.1 Limitations and strengths

12.1.1 Limitations

- **Single-subject data** → all 6 sessions are from one driver. A classifier trained on one person’s steering may learn that individual’s fatigue pattern rather than drowsiness in general. The results should be framed as a within-subject proof of concept.
- **Mouse steering** → while the mouse provides continuous steering, the motor patterns differ from a physical steering wheel. The `input_device` column is logged for transparency.
- **Small number of sessions** → with 3 sessions per class, the per-session majority vote can partly reflect “which session” rather than drowsiness. More sessions per class would strengthen the result.
- **ML fires after overcorrection** → the ML model’s dominant features (`steer_vel_sd`) require an overcorrection to have already occurred. The Tier 1 inactivity detector mitigates this but is a simpler heuristic.

12.1.2 Strengths

- **Honest evaluation** → leave-one-session-out cross-validation prevents data leakage and produces trustworthy metrics. The 79.3% accuracy and 0.850 ROC-AUC are genuine out-of-sample numbers.
- **Steering-only approach** → requires no additional sensors, making the system applicable to any vehicle with electronic steering.
- **Interpretable model** → logistic regression coefficients map directly to the drowsiness literature (steering velocity variability, LF/HF ratio shift), providing clinical and engineering justification.
- **Two-tier warning design** → the fast rule-based trigger catches microsleep onset within 3 seconds, while the ML model confirms sustained drowsiness patterns.

- **6/6 session-level accuracy** → despite modest window-level accuracy, every session is correctly classified by majority vote, demonstrating that the system reliably detects drowsiness at the timescale that matters for safety.

12.1.3 Future developments

- **Multi-subject validation** → collecting data from multiple drivers to test generalisation
- **Physical steering wheel** → using a force-feedback wheel for more realistic steering dynamics
- **Additional drowsiness sessions** → increasing the number of sessions per class for more robust cross-validation
- **Camera fusion** → combining steering features with PERCLOS (eye closure) from a webcam for a multi-modal detector
- **Simulated drowsy driver model** → using a control-loop degradation model (increased latency, reduced gain, microsleep dropout) to generate unlimited synthetic drowsy data for pre-training.

12.2 Closing remarks

While the developed system is a single-subject prototype, it demonstrates that drowsiness can be reliably detected from the steering signal alone using interpretable machine learning and a two-tier warning architecture. The 6/6 session-level accuracy with honest leave-one-session-out evaluation, combined with the real-time MQTT integration, provides a solid foundation for a more comprehensive drowsiness detection ADAS.

Bibliography

- [1] European Commission, *Fatigue*, European Road Safety Observatory, 2021.
- [2] European Transport Safety Council, *The Role of Driver Fatigue in Commercial Road Transport Crashes*, ETSC Pin Flash Report, 2021.
- [3] O. Nakayama, T. Futami, T. Nakamura, E.R. Boyle, *Development of a Steering Entropy Method for Evaluating Driver Workload*, SAE Technical Paper 1999-01-0892, 1999.
- [4] W.W. Wierwille, L.A. Ellsworth, S.S. Wreggit, R.J. Fairbanks, C.L. Kirn, *Research on Vehicle-Based Driver Status/Performance Monitoring*, NHTSA Technical Report DOT HS 808 247, 1994.
- [5] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, V. Koltun, *CARLA: An Open Urban Driving Simulator*, Conference on Robot Learning (CoRL), 2017.